



## **2.3 Sincronização de Threads**

*Celso Paím*



## 2.3. Sincronização entre threads

- A palavra-chave `synchronized` em Java controla o acesso a recursos compartilhados entre várias *threads*, garantindo que somente uma *thread* possa executar um bloco ou método sincronizado por vez, evitando a interferência de *threads* e garantindo a consistência da memória.
- `synchronized` pode ser aplicada a métodos ou blocos dentro de métodos. Quando um método ou bloco é declarado como sincronizado, um bloqueio (*lock*) é obtido no objecto especificado, e outras *threads* são impedidas de acessar o código sincronizado até que o bloqueio seja libertado.
- Com essa palavra podem se criar os chamados monitores, semáforos ou *mutex* (mutuamente exclusivos), constituindo um *lock* (trinco/cadeado) que só pode ser destrancado por quem o trancou.



## 2.3. Sincronização entre threads

Imagine que se tem  $\text{saldo} = 500$  e hajam duas *threads* a fazerem simultaneamente  $\text{saldo} = \text{saldo} + 100$  **sem estarem sincronizadas**:

- 1)- Thread 1 lê o saldo (500)
- 2)- Thread 2 lê o saldo (500)
- 3)- Thread 1 escreve em saldo (600)
- 4)- Thread 2 escreve em saldo (600)



**Resultado: no final saldo será igual a 600.**



## 2.3. Sincronização entre threads

Porém se o acesso for **sincronizado**:

- 1)- Thread 1 lê o saldo (500) e é travado o acesso ao recurso (variável `saldo`. Todas as *threads* que tentarem ficarão no estado BLOCKED)
- 2)- Thread 1 escreve em saldo (600)
- 3)- Thread 1 libera o recurso
- 4)- Thread 2 lê o saldo (600) e é travado o acesso ao recurso (600)
- 5)- Thread 2 escreve em saldo (700)
- 6)- Thread 2 libera o recurso



**Resultado: no final saldo será igual a 700.**



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Abril/2026

// Demonstrar o conceito de lock e de sincronizacao entre threads

```
public class TesteSyncThreads {  
    private static final Object LOCK = new Object();  
  
    public static void main( String[] args ) throws InterruptedException {  
  
        Thread t1 = new Thread( () -> {  
            synchronized ( LOCK ) {  
                System.out.println( "T1 obteve o recurso LOCK" );  
                try {  
                    Thread.sleep( 5000 );  
                }  
                catch ( InterruptedException e ) {  
                }  
                System.out.println( "T1 libertou o LOCK" );  
            }  
        }, "Thread-1" );
```



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

```
Thread t2 = new Thread( () -> {  
    System.out.println( "T2 tentando obter o LOCK..." );  
    synchronized ( LOCK ) { // Aqui ela vai ficar BLOCKED  
        System.out.println( "T2 conseguiu o recurso LOCK e esta a ser executada" );  
    }  
}, "Thread-2" );
```

```
t1.start();  
Thread.sleep( 100 ); // Garante que t1 pegue o LOCK primeiro  
t2.start();  
System.out.println( "Estado da T2 no inicio: " + t2.getState() );
```

```
Thread.sleep( 500 ); // dar tempo pra t2 ficar blocked  
System.out.println( "Estado da T2: " + t2.getState() ); // BLOCKED
```

```
}  
}
```



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

## 2.3. Sincronização entre threads

- Apenas usar `synchronized` perante múltiplas *threads* que lêem e escrevem no mesmo recurso. Se só leem, é desnecessário, até porque afecta a performance do algoritmo (o tempo que as *threads* ficam paradas bloqueadas esperando a libertação do recurso).
- Minimizar o código sincronizado: os blocos sincronizados devem ser tão curtos quanto possível para reduzir a contenção e melhorar o desempenho.
- Evitar a sincronização aninhada: Os blocos sincronizados aninhados podem levar a *deadlocks*.
- Atenção aos fenómenos de **starvation**: ocorre quando uma ou mais *threads* estão impedidas de obter acesso a um recurso, não podendo, conseqüentemente, progredir e de **deadlock** (*starvation* ao máximo), que ocorre quando duas ou mais *threads* estão à espera numa condição que não pode ser satisfeita.



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

## 2.3. Sincronização entre threads

- Para variáveis que são acessadas por várias threads, mas que não exigem sincronização complexa, pode ser utilizada a palavra-chave `volatile` para garantir a visibilidade correcta.
- Certificar sempre de que os recursos compartilhados estejam devidamente sincronizados para manter a segurança da *thread* e evitar a corrupção de dados.
- Sincronizar métodos estáticos para impor o acesso sincronizado no nível da classe, garantindo que apenas uma *thread* possa executar o método estático em todas as instâncias.
- Para sincronização mais complexas, considerar o uso de coleções simultâneas do pacote `java.util.concurrent`, como `ConcurrentHashMap`, que oferece sincronização integrada, ou outras classes como `AtomicInteger` ou `ReentrantLock` (que também permite criar mutex personalizados).





# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

## 2.3.1 Sincronização entre threads produtoras e consumidoras

- O problema produtor-consumidor (ou **Bounded Buffer Problem**) é um clássico cenário de concorrência onde *threads* produtoras geram dados e as consumidoras os processam, compartilhando um buffer limitado.
- Threads Produtoras: criam dados e os colocam em um buffer compartilhado. Se o buffer estiver cheio, a produtora deve aguardar (`wait`).
- Threads Consumidoras: retiram dados do buffer para processamento. Se o buffer estiver vazio, a consumidora deve aguardar (`wait`). Sincronização:
- A sincronização é essencial para garantir que o produtor não preencha um buffer cheio e o consumidor não leia um buffer vazio. Faz-se recurso a mecanismos como *locks/mutex* (trincos) e coordenação do acesso (com métodos como `wait` ou `notify`), evitando que dados sejam perdidos ou sobrescritos.



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

#### Buffer.java

// Autores: Deitel & Deitel

// Buffer interface specifies methods called by Producer and Consumer.

```
public interface Buffer {  
    public void set( int value );  
    public int get();  
}
```



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

#### SynchronizedBuffer.java

// Autores: Deitel & Deitel

// SynchronizedBuffer synchronizes access to a single shared integer.

```
public class SynchronizedBuffer implements Buffer {
    private int buffer = -1;           // shared by producer and consumer threads
    private int occupiedBufferCount = 0; // count of occupied buffers

    // place value into buffer
    public synchronized void set( int value ) {
        String name = Thread.currentThread().getName();

        // while there are no empty locations, place thread in waiting state
        while ( occupiedBufferCount == 1 ) {
            // output thread information and buffer information, then wait
            try {
                System.err.println( name + " tries to write." );
                displayState( "Buffer full. " + name + " waits." );
                wait();
            }
            catch ( InterruptedException exception ) {
                exception.printStackTrace();
            }
        }
    }
}
```



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

```
buffer = value; // set new buffer value
// indicate producer cannot store another value until consumer retrieves current buffer value
++occupiedBufferCount;
displayState( name + " writes " + buffer );
notify(); // tell waiting thread to enter ready state
} // end method set and releases lock on SynchronizedBuffer

// return value from buffer
public synchronized int get() {
    String name = Thread.currentThread().getName();

    // while no data to read, place thread in waiting state
    while ( occupiedBufferCount == 0 ) {
        // output thread information and buffer information, then wait
        try {
            System.err.println( name + " tries to read." );
            displayState( "Buffer empty. " + name + " waits." );
            wait();
        }
        catch ( InterruptedException exception ) {
            exception.printStackTrace();
        }
    }
}
```



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

// indicate that producer can store another value because consumer just retrieved buffer value  
--occupiedBufferCount;

displayState( name + " reads " + buffer );

notify(); // tell waiting thread to become ready to execute

return buffer;

} // end method get and releases lock on SynchronizedBuffer

// display current operation and buffer state

public void displayState( String operation ) {

StringBuffer outputLine = new StringBuffer( operation );

outputLine.setLength( 40 );

outputLine.append( buffer + "\t\t" + occupiedBufferCount );

System.err.println( outputLine );

System.err.println();

}

}



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

#### Producer.java

// Autores: Deitel & Deitel

// Producer's run method controls a thread that stores values from 1 to 4 in sharedLocation.

```
public class Producer extends Thread {
    private Buffer sharedLocation;

    public Producer( Buffer shared ) {
        super( "Producer" );
        sharedLocation = shared;
    }

    // store values from 1 to 4 in sharedLocation
    public void run() {
        for ( int count = 1; count <= 4; count++ ) {
            try {
                Thread.sleep( ( int ) ( Math.random() * 3001 ) ); // sleep 0 to 3 sec, then place value in Buffer
                sharedLocation.set( count );
            }
            catch ( InterruptedException exception ) { exception.printStackTrace(); }
        }
        System.err.println( getName() + " done producing.\nTerminating " + getName() + "." );
    }
}
```



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

#### Consumer.java

// Autores: Deitel & Deitel

// Consumer's run method controls a thread that loops four times and reads a value from sharedLocation each time.

```
public class Consumer extends Thread {  
    private Buffer sharedLocation;  
  
    public Consumer( Buffer shared ) {  
        super( "Consumer" );  
        sharedLocation = shared;  
    }  
  
    // read sharedLocation's value four times and sum the values  
    public void run() {  
        int sum = 0;  
        for ( int count = 1; count <= 4; count++ ) {  
            // sleep 0 to 3 seconds, read value from Buffer and add to sum  
            try {  
                Thread.sleep( ( int ) ( Math.random() * 3001 ) );  
                sum += sharedLocation.get();  
            }  
        }  
    }  
}
```



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

```
    catch ( InterruptedException exception ) {  
        exception.printStackTrace();  
    }  
}
```

```
System.err.println( getName() + " read values totaling: " + sum + ".\nTerminating " + getName() +  
".");  
}  
}
```





# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

#### SharedBufferTest2.java

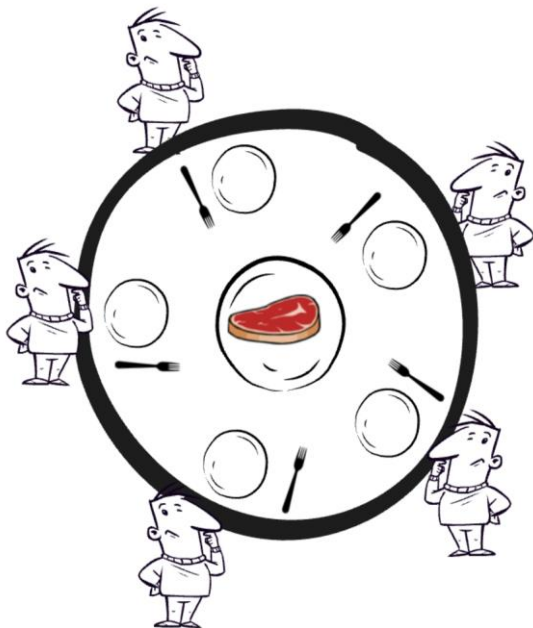
// Autores: Deitel & Deitel

// SharedBufferTest2 creates producer and consumer threads.

```
public class SharedBufferTest2 {  
    public static void main( String [] args ) {  
        SynchronizedBuffer sharedLocation = new SynchronizedBuffer();  
  
        StringBuffer columnHeads = new StringBuffer( "Operation" );  
        columnHeads.setLength( 40 );  
        columnHeads.append( "Buffer\t\tOccupied Count" );  
        System.err.println( columnHeads );  
        System.err.println();  
        sharedLocation.displayState( "Initial State" );  
  
        Producer producer = new Producer( sharedLocation );  
        Consumer consumer = new Consumer( sharedLocation );  
        producer.start();  
        consumer.start();  
    }  
}
```

## 2.3.1 Sincronização entre threads produtoras e consumidoras

- Investigar as possíveis soluções e implementação em Java do problema clássico do **Jantar dos Filósofos**.



Este problema proposto por Edsger W. Dijkstra em 1965, é um problema de sincronização que consiste em 5 filósofos que passam a vida pensando e comendo.

Eles partilham de uma mesa redonda rodeada por 5 lugares pertencendo a cada filósofo. No centro da mesa encontra-se uma bandeja contendo um alimento e 5 garfos (hashi) na mesa, sendo um para cada filósofo.

Quando um filósofo pensa, ele não interage com os outros.

De tempos em tempos, cada filósofo fica com fome e tenta apanhar os dois garfos que estão mais próximos (os garfos que estão ou à esquerda ou à direita). O filósofo apenas pode apanhar um garfo de cada vez, além de não pode apanhar um garfo se este estiver na mão do vizinho. Quando um filósofo esfomeado tiver 2 garfos ao mesmo tempo ele come sem largar os garfos.

Apenas quando acaba de comer, o filósofo pousa os garfos na mesa, libertando-os e começa a pensar de novo.

<https://www.youtube.com/watch?v=eEDstT6cAD8>



# Universidade Católica de Angola

## Faculdade de Engenharia Informática

### Sistemas Distribuídos e Paralelos I

